EAST WEST UNIVERSITY

A/2,Jahurul Islam Avenue Jahurul Islam city,Aftabnagar Dhaka-1212,Bangladesh

Cse Department
Fall-2025
Project Report
Penalty 3D Shootout

Submitted To:

COURSE INSTRUCTOR : Dr.md.Tauhid Bin Iqbal (Assistant Professor)
COURSE NAME : Computer Graphics
COURSE CODE : CSE 420
SECTION : 01
DATE : 20th,April 2026

Submitted by:

NAME : Siam Ahmed
STUDENT ID : 2022-2-60-074
NAME : Mehedi Hasan Tapu
STUDENT ID : 2021-2-60-154
NAME : Khairul Alam Bin Jahangir
STUDENT ID : 2022-3-60-313
NAME : Ahsiul Karim Audri
STUDENT ID : 2022-3-60-074

1. Abstract

Penalty Blitz 3D is an interactive, real-time 3D football penalty shootout simulation built using C++ with OpenGL and FreeGLUT. The application renders a full-fledged virtual football stadium complete with animated fans, a dynamic sky with moving clouds and birds, a detailed pitch with markings, corner flags, and a goal post with net. Two players — representing Argentina and Brazil — compete through two complete rounds of five penalty kicks each, alternating between the roles of shooter and goalkeeper.

This report provides a comprehensive technical analysis of the project's architecture, the 3D graphics algorithms and mathematical models employed, the system's complete program flow, and the significance of each user control. The project demonstrates core concepts in real-time 3D rendering, scene graph management, collision detection, particle systems, and interactive gameplay design within a fixed-function OpenGL pipeline.

1.1 Introduction

Real-time 3D graphics have transformed the landscape of interactive applications, from professional simulation tools to entertainment software. OpenGL, as a cross-platform graphics API, offers a versatile and mature framework for constructing complex visual environments entirely in software. This project leverages OpenGL's fixed-function pipeline in conjunction with FreeGLUT — a portable windowing and input toolkit — to realise a two-player, head-to-head penalty shootout experience.

The simulation places two human players at opposite ends of a 3D rendered football stadium. During each round, one player controls a shooting cursor to aim and fire a penalty kick while the opponent manoeuvres the goalkeeper horizontally to attempt a save. The scene is enriched with ambient crowd animations, atmospheric weather effects, and real-time visual feedback — all rendered at approximately 60 frames per second using a 16 ms timer callback loop.

1.2 Problem Statement

Building an interactive 3D sports simulation in native C++ with a fixed-function OpenGL pipeline presents several interlinked technical challenges:

- Constructing a visually convincing 3D stadium environment using only primitive geometric shapes (spheres, cylinders, cubes, and quads) — without access to mesh importers or texture mapping.
- Implementing physically plausible ball trajectory physics and real-time collision detection between the ball, the goal posts, and the goalkeeper model.
- Coordinating a multi-phase game loop (menu, round one, halftime, round two, result) with clean state transitions and persistent score tracking.
- Animating multiple independent scene elements — fans, clouds, birds, corner flags, and star particles — simultaneously without degrading frame rate.
- Designing a two-player local control scheme that is intuitive and responsive for simultaneous input from both players on the same

keyboard.

1.3 Motivation

The primary motivation behind this project is to explore and demonstrate the depth of capability achievable with OpenGL's legacy fixed-function pipeline, which remains widely taught in computer graphics curricula. Rather than relying on modern shader-based pipelines or game engine abstractions, the project intentionally employs classic OpenGL constructs — glBegin/glEnd, GLU quadrics, glLightfv, and gluPerspective — to illustrate foundational 3D rendering concepts in their most transparent form.

Additionally, the project is motivated by the desire to combine academic graphics theory (lighting, projection, depth testing, blending) with practical game design (state machines, player input handling, feedback systems). Football — and penalty shootouts in particular — provide an ideal domain because the gameplay is simple enough to implement within a constrained scope while being sufficiently engaging to motivate a complete, polished experience.

1.4 Objectives

The key objectives of this project are as follows:

1. Construct a fully 3D football stadium environment using OpenGL primitives, complete with pitch markings, goal posts, tiered stadium seating, a skybox, clouds, and birds.
2. Implement a two-round penalty shootout game loop supporting two local players alternating between shooter and goalkeeper roles.
3. Develop a real-time ball trajectory simulation with physically

motivated velocity components and rotational animation.

4. Create a goalkeeper AI-assisted dive animation triggered at the moment of a shot, with human override through keyboard input.

5. Design and render a responsive HUD displaying live scores, shot indicators, team labels, and feedback messages.

6. Implement a crowd celebration system that animates fans in the shooting team's section upon a goal.

7. Integrate atmospheric animation elements including moving clouds, a bird flock, and waving corner flags to enhance environmental realism.

8. Deploy a particle burst (star) system to provide visual reinforcement of goal-scoring events.

2. Background Studies

2.1 OpenGL Fixed-Function Pipeline

OpenGL (Open Graphics Library) is a cross-platform API for rendering 2D and 3D vector graphics. The fixed-function pipeline, used in this project, processes geometry through a predetermined sequence of stages: vertex transformation, lighting calculation, rasterisation, and fragment operations. While modern OpenGL replaces these fixed stages with programmable vertex and fragment shaders, the fixed-function approach remains pedagogically valuable because it exposes the underlying mathematics directly to the programmer through API calls such as glLightfv,

glColorMaterial, and glShadeModel.

This project makes extensive use of the fixed-function pipeline's built-in Gouraud shading (GL_SMOOTH), two configurable light sources (GL_LIGHT0 and GL_LIGHT1), the depth buffer (GL_DEPTH_TEST), and the alpha blending stage (GL_BLEND). The GLU library supplements OpenGL with higher-level utilities including gluPerspective for projection setup and GLUquadric objects for generating cylindrical and spherical geometry with smooth normals.

2.2 FreeGLUT Windowing and Event Model

FreeGLUT (Free OpenGL Utility Toolkit) provides platform-independent window management, OpenGL context creation, and an event-driven callback architecture. The application registers six GLUT callbacks: glutDisplayFunc (rendering), glutReshapeFunc (viewport), glutKeyboardFunc (key-down), glutKeyboardUpFunc (key-up), glutSpecialFunc (arrow keys), and glutTimerFunc (game loop timer). The timer callback fires every 16 milliseconds, approximating a 62.5 Hz update rate, and calls glutPostRedisplay to trigger a render pass after each update.

2.3 Scene Graph Architecture

Although a formal scene graph data structure is not employed, the project organises its rendering into a logical hierarchy of draw calls within the display() function. Each major scene component has a dedicated draw function: drawSky(), drawCrowd(), drawPitch(), drawGoalPost(), drawGoalkeeper(), drawBall(), drawAimCrosshair(), and drawHUD(). This modular organisation mirrors the layered

approach of scene graph-based engines, with background elements rendered before foreground objects, and 2D HUD elements rendered last after the 3D world geometry.

2.4 Lighting and Shading Models

The project configures two OpenGL light sources to simulate a daytime stadium environment. GL_LIGHT0 acts as the primary sunlight: a warm, high-intensity positional light placed above and behind the camera at (5, 20, 15). GL_LIGHT1 provides a cool ambient fill light from (−5, 5, 25), simulating the diffuse sky light that fills shadow regions. Both lights use the Phong reflection model's ambient, diffuse, and specular components, with GL_COLOR_MATERIAL enabling each geometric primitive to inherit its surface colour from the active glColor call.

2.5 Alpha Blending and Transparency

Multiple scene elements in this project require semi-transparency: goal net geometry, the aim crosshair, HUD panels, cloud spheres, and celebration banners. OpenGL's blending equation is configured with GL_SRC_ALPHA and GL_ONE_MINUS_SRC_ALPHA (the standard Porter-Duff 'over' compositing operator), which interpolates the source fragment's colour with the destination framebuffer colour based on the source alpha value. Transparent elements are carefully rendered after all opaque geometry to ensure correct depth-sorted compositing.

2.6 Particle Systems

Particle systems are a well-established technique in real-time graphics for simulating phenomena such as explosions, fire, sparks,

and confetti. This project implements a simple 2D particle burst triggered upon a goal: 80 point-sprite particles are spawned at the screen centre with randomised initial velocities in all directions. Each particle carries its own colour (randomised RGB), position, velocity, and life value. The update loop applies a downward gravitational acceleration to each particle's Y-velocity (−0.08 units/frame) and decrements its life value by 0.012 per frame, providing a natural arc and fade-out effect.

2.7 State Machine Game Design

The game's flow is governed by a four-state finite state machine encoded in the GameState enumeration: STATE_MENU, STATE_PLAYING, STATE_HALFTIME, and STATE_RESULT. State transitions are triggered by player input (ENTER, R keys) and by timer callbacks scheduled with glutTimerFunc after a round completes. Each state determines which draw functions are invoked in display() and which input handlers are active, providing clean separation of concerns between gameplay, UI, and transition logic.

3. Methodology

3.1 Graphics Algorithms and Mathematical Models Used

The simulation employs several classical computer graphics algorithms and mathematical formulations, each chosen to balance visual fidelity with the computational constraints of a fixed-function, CPU-driven rendering loop.

3.2 Ball Trajectory: Linear Interpolation with Gravitational Bias

When a shot is fired, the ball's velocity is computed by decomposing the displacement vector from the ball's current position to the aim target into three orthogonal velocity components. The Z-component is fixed at −0.5 units/frame (towards the goal). The X and Y components are derived by dividing the respective displacements by the number of frames required to traverse the Z-distance:

```
steps = |ballZ - GOAL_Z| / 0.5
ballDX = (aimX - ballX) / steps    ballDY = (aimY - ballY) / steps
```

This linear interpolation model ensures the ball reaches the aim point exactly as it crosses the goal line, producing predictable shooting behaviour. Rotational animation is added by incrementing ballRotX by 8° and ballRotZ by 3° per frame, giving the appearance of a spinning ball without requiring quaternion-based rotation.

3.3 Goalkeeper Dive: Parametric Lerp Animation

Upon a shot being fired, triggerKeeperDive() records the goalkeeper's current position as the dive origin. The dive animation uses linear parametric interpolation (lerp) over a 0.5-second (30-frame) window:

```
t = keeperDiveTimer / 0.5,   kx = keeperX + (keeperDiveX - keeperX) * t
```

The dive target is implicitly determined by the keeper's last known

position before the shot, which is influenced by the human defender's keyboard input. This design means the defender must predict the shot direction and pre-position accordingly, creating genuine tactical gameplay.

3.4 Collision Detection: Euclidean Distance Test

Goal detection uses a bounding box test: the ball must be within the goal's X and Y extents (GOAL_LEFT to GOAL_RIGHT, 0 to GOAL_TOP) when its Z coordinate crosses the goal line (GOAL_Z ± 0.5). A save is then determined by a circular proximity test using the Euclidean distance formula between the ball's centre and the keeper's interpolated position at the moment of crossing:

```
dist = sqrt((ballX - kx)^2 + (ballY - ky)^2)   <   1.1 units
```

The 1.1-unit threshold corresponds approximately to the goalkeeper model's half-width, providing a fair and spatially accurate save detection radius.

3.5 Sinusoidal Fan Celebration Animation

Fan celebration uses sinusoidal oscillation to drive arm waving and body bobbing. Each fan's animation phase is offset by a pre-generated random value (fanCelebPhase), producing an organic, staggered wave effect across the crowd:

```
bob = sin(phase) * 0.07      armAngle = sin(phase + PI/2) * 22.0
```

The phase value is incremented by 4.0 * deltaTime when celebrating

is active, producing a wave frequency of approximately 2 Hz — matching the natural cadence of crowd chanting.

3.6 Corner Flag Wave — Sinusoidal Quad Deformation

Corner flags are rendered as a deformable quadrilateral whose upper vertices are displaced sinusoidally:

```
wave = sin(flagWave) * 0.20      wave2 = sin(flagWave + 1.1) * 0.13
```

The phase offset of 1.1 radians between the two upper vertices creates a billowing effect that simulates wind-driven cloth deformation without requiring a full cloth simulation.

3.7 Idle Ball Bob — Simple Harmonic Motion

While the ball is stationary, a gentle vertical oscillation is applied using simple harmonic motion to give the scene a sense of life:

```
ballY = 0.55 + 0.06 * sin(idleTimer)
```

The idleTimer is incremented by 0.05 radians per frame (~3°/frame), yielding a bob frequency of roughly 0.5 Hz.

3.8 Technology Stack

Component	Technology	Purpose
Programming Language	C++ (C++11)	Core application logic, game loop, state management
Graphics API	OpenGL (Legacy Fixed-Function)	3D scene rendering, lighting, blending, depth testing
Window & Input Toolkit	FreeGLUT	Window creation, event callbacks, timer loop
Geometry Utilities	GLU (OpenGL Utility Library)	Quadric primitives (cylinders, spheres), perspective projection
Math Library	C standard library (cmath)	Trigonometric functions for animation and physics
Build Platform	Windows (MSVC / MinGW)	Target OS, GLUT header inclusion via windows.h
Rendering Mode	Immediate Mode (glBegin/glEnd)	Direct vertex submission per frame

4. Implementation

4.1 Module Overview

The application is implemented in a single C++ translation unit (~1,000 lines) with a clear functional decomposition into thematic modules:

Module	Key Functions	Responsibility
Initialisation	init(), initClouds(), initBirds(), initCelebPhases()	Seeds RNG, configures OpenGL state, populates initial scene data
Scene Geometry	drawPitch(), drawGoalPost(), drawCornerFlags()	Renders static field and goal infrastructure
Characters	drawGoalkeeper(), drawBall(), drawAimCrosshair()	Renders dynamic in-game objects
Environment	drawSky(), drawClouds(), drawBirds()	Renders sky gradient, cloud blobs, bird flock
Crowd System	drawCrowd(), drawStadiumSeats(), drawSeatedFan()	Renders tiered seating, fan figures with team colours
HUD / UI	drawHUD(), drawShotIndicators(), drawStars()	Renders 2D overlays, feedback, particle effects
Screens	drawMenuScreen(), drawHalftimeScreen(), drawResultScreen()	Full-screen overlay UIs for each game phase
Physics / Logic	shoot(), update()	Ball trajectory, collision, score tracking, state transitions
Input	keyboard(), keyboardUp(), specialKeys()	Keyboard event handlers for both players
Lighting	setupLighting()	Configures two OpenGL light sources per frame

4.2 Rendering Pipeline Per Frame

Each rendered frame follows this sequence within display():

9. Clear colour and depth buffers (glClear).
10. Set the camera view matrix via gluLookAt (eye at (0,6,26), target at (0,2.5,0)).
11. Call setupLighting() to position light sources in world space.
12. Render sky gradient, clouds, and birds (background, depth writes suppressed by far-plane distance).
13. Render crowd and stadium structure.
14. Render pitch, goal post, goalkeeper, corner flags.
15. Render ball and aim crosshair.
16. Enter 2D orthographic mode (enter2D) to render HUD, shot indicators, and star particles.
17. Overlay halftime or result screen if applicable.
18. Call glutSwapBuffers to present the double-buffered frame.

4.3 Two-Player Input Mapping

Player	Round 1 Role	Controls	Round 2 Role	Controls
P1 — Argentina	Shooter	LEFT/RIGHT arrows = Aim horizontal, SPACE = Shoot	Goalkeeper	A/D keys = Move keeper left/right
P2 — Brazil	Goalkeeper	A/D keys = Move keeper left/right	Shooter	LEFT/RIGHT arrows = Aim, SPACE = Shoot

The aim crosshair is clamped within the goal's lateral extents ([GOAL_LEFT+0.5, GOAL_RIGHT−0.5]) to prevent obviously illegal shots. The goalkeeper is clamped to [−4.2, +4.2] on the X-axis, matching the inner post positions.

4.4 Shot Indicator System

Five circular indicators are rendered at the top-right of the HUD, one per shot. Each circle is drawn as a GL_TRIANGLE_FAN for the fill and a GL_LINE_LOOP for the outline. The fill colour encodes the result: black (pending), green (goal), red (saved/missed). A checkmark, X mark, or small dot is drawn inside the circle to provide an iconic reading of the result at a glance.

4.5 Stadium and Crowd Architecture

The stadium is constructed from three tiered stand sections: a back stand (5 rows of 28 fan columns), a left side stand (5 rows x 12 columns, Argentina colours), and a right side stand (5 rows x 12 columns, Brazil colours). Each tier consists of a concrete seat platform quad and a vertical riser face. Fan figures are placed at regular intervals atop each tier at Y positions that step upward with each row, simulating a real stadium rake.

Each fan model is composed of six to nine glutSolidCube and glutSolidSphere primitives, coloured with jersey and trouser colours selected from team-specific palettes. When celebrating, fans switch from a seated pose to a standing pose with sinusoidally waving arms.

5. Program Flowchart

The following textual flowchart describes the complete execution path of the application, from startup through to termination.

APPLICATION FLOWCHART

Step	Action / Decision	Next Step
START	main() — glutInit, create window, call init()	MENU STATE
MENU STATE	drawMenuScreen() — display controls and team info	Wait for ENTER key
ENTER pressed	initClouds(), initBirds(), initCelebPhases(), startRound(TEAM_ARGENTINA)	ROUND 1 — PLAYING STATE
PLAYING STATE	update() fires every 16 ms — advance game logic	Each update cycle below
Update: No shot active	Idle ball bob, keeper moves per A/D input, update clouds/birds/flags	Repeat loop
SPACE pressed	shoot() — compute ballDX/DY/DZ, triggerKeeperDive(),	Ball in flight

	decrement shotsLeft	
Ball in flight	Each frame: advance ballX/Y/Z, rotate ball, advance keeper dive	Check Z position
Ball at GOAL_Z	Bounding box test: inX && inY?	Yes: save test / No: OUT
OUT	feedbackMsg = 'OUT!', result = MISSED, record shotResult[i] = -1	Reset ball
Save test (Euclidean dist)	dist < 1.1 ?	Yes: SAVED / No: GOAL
SAVED	feedbackMsg = 'SAVED!', celebrating = defender team, result = -1	Reset ball
GOAL	score++, spawnStars(), triggerCelebration(shootingTeam), result = +1	Reset ball
shotsLeft == 0?	No: continue loop / Yes: schedule halftime/result transition	Yes: advance state
Round 1 ends (Argentina)	argScore = score, 2.2 s delay then STATE_HALFTIME	HALFTIME STATE
HALFTIME STATE	drawHalftimeScreen() — show ARG score, prompt Round 2	Wait for ENTER
ENTER pressed	startRound(TEAM_BRAZIL) — swap shooter/keeper roles	ROUND 2 — PLAYING STATE

Round 2 ends (Brazil)	braScore = score, 2.2 s delay then STATE_RESULT	RESULT STATE
RESULT STATE	drawResultScreen() — compare scores, display winner	R = restart, ESC = quit
R pressed	Reset argScore/braScore, startRound(TEAM_ARGENTINA)	Back to ROUND 1
ESC pressed	exit(0)	END

6. Workflow

6.1 Development Workflow

The development of Penalty Blitz 3D followed an iterative, module-by-module construction workflow:

19. Environment Scaffolding: The project began with setting up the GLUT window, OpenGL context, and a basic perspective projection. The pitch, skybox, and goal post were constructed first to establish the spatial reference frame.

20. Character Construction: The goalkeeper model was assembled from primitives, followed by the ball model with its hexagonal patch detailing. Jersey colour logic was added to support team switching.

21. Physics and Shooting: The ball velocity calculation and trajectory system were implemented and tuned. The aim crosshair was added

as a visual aid for the shooter.

22. Collision and Scoring: Bounding box goal detection and Euclidean save detection were integrated. The five-shot round structure and score tracking variables were added.

23. HUD and Feedback: The 2D HUD overlay system (enter2D/leave2D orthographic switch) was implemented. Shot indicators, score panel, and feedback text were integrated.

24. Crowd and Animation: The tiered stadium seating and fan figure system were constructed. Celebration animations, sinusoidal flag waving, cloud movement, and bird flocking were added.

25. Multi-Round Flow: The four-state FSM, halftime screen, and result screen were implemented. The two-round role-swap logic was finalised.

26. Polish and Tuning: Particle star burst, alpha-blended celebration banners, per-fan celebration phase randomisation, and lighting tweaks were applied to reach the final visual quality level.

6.2 Per-Frame Execution Workflow

The per-frame workflow executed every 16 ms by the glutTimerFunc callback proceeds as follows:

- Advance cloud X positions; wrap clouds that drift past −60 units back to +60.
- Advance bird flock X position and flap phase accumulators.
- Increment flagWave accumulator.
- Decrement celebTimer; clear celebration flags when it reaches zero.

- If no shot active: apply idle ball bob via sinf(idleTimer).
- If keeper movement keys held: update keeperX within clamp bounds.
- If shot active: advance ball position, check goal plane crossing, resolve goal/save/miss.
- If shotsLeft == 0 after resolution: schedule state transition via glutTimerFunc.
- Decrement feedbackTimer; advance star particle positions and fade life values.
- Call glutPostRedisplay to queue the next frame render.

7. Importance of the Controls

7.1 View and Display Controls

The application uses a fixed perspective camera established by gluLookAt(0, 6, 26, 0, 2.5, 0, 0, 1, 0). This places the observer 6 units above the field and 26 units back from centre, tilted slightly downward toward the goal. This viewpoint was chosen after iterative testing to provide:

- Full visibility of the pitch, goal, and goalkeeper within the frustum without requiring camera movement.
- A sense of depth and perspective consistent with a broadcast-style side-on/front view of a penalty kick.
- Sufficient vertical field of view (45°, set via gluPerspective) to

show the upper corners of the goal — the most challenging shooting targets.

The reshape() callback maintains the correct aspect ratio on window resize by recalculating the projection matrix, ensuring the scene does not distort on non-standard display configurations.

7.2 Movement and Camera Controls

All player interaction is handled through the keyboard, divided by function and round:

Shooter Controls (LEFT / RIGHT Arrow Keys + SPACE)

The arrow keys adjust the aim crosshair's X position in 0.32-unit increments, clamped within the goal width. This granularity was calibrated to allow both precise corner shots and rapid cross-goal movement within the typical reaction time of a goalkeeper. The crosshair is rendered as a red circle with crosshairs at the goal plane depth (z = GOAL_Z + 0.05), providing intuitive visual feedback of where the ball will be directed. SPACE triggers the shoot() function, which commits the aim position as the shot target and initiates the ball trajectory.

The importance of these controls lies in their simplicity: the shooter has only one degree of freedom (horizontal aim), keeping the interaction accessible while preserving strategic depth through timing and deception.

Goalkeeper Controls (A / D Keys)

The A and D keys set boolean flags (p2MoveLeft, p2MoveRight) that are polled every frame in the update loop, moving the keeper at 0.17 units/frame. This continuous-movement design (as opposed to a single keypress jump) means a defender must hold a direction key to reposition, creating a more nuanced guessing game: committing too early allows the shooter to aim the opposite way; waiting too long means insufficient time to reach the corner.

The keeper is constrained to the goal's lateral extent ([−4.2, +4.2]), preventing trivially easy saves from outside the post line while allowing the full goal face to be covered.

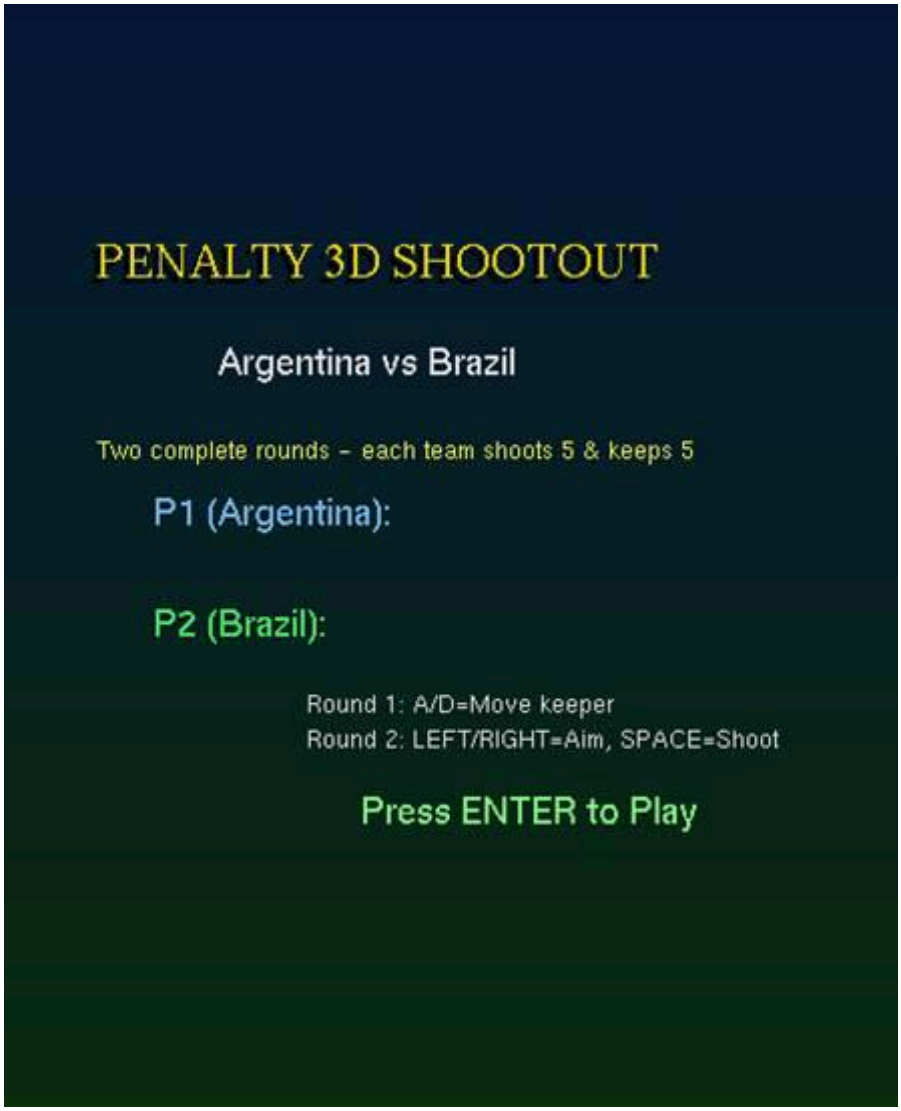
Global Controls (ENTER / R / ESC)

Key	Context	Action	Design Rationale
ENTER	Menu / Halftime	Start Round 1 / Start Round 2	Universal 'confirm' key; separates game phases clearly
R	Playing / Halftime / Result	Restart full match	Provides instant replayability without navigating menus
ESC	Any state	Exit application	Standard convention for windowed applications
SPACE	Playing (shooter's turn)	Fire the penalty kick	Spacebar is the most natural 'action' key; prevents accidental shots by requiring deliberate press

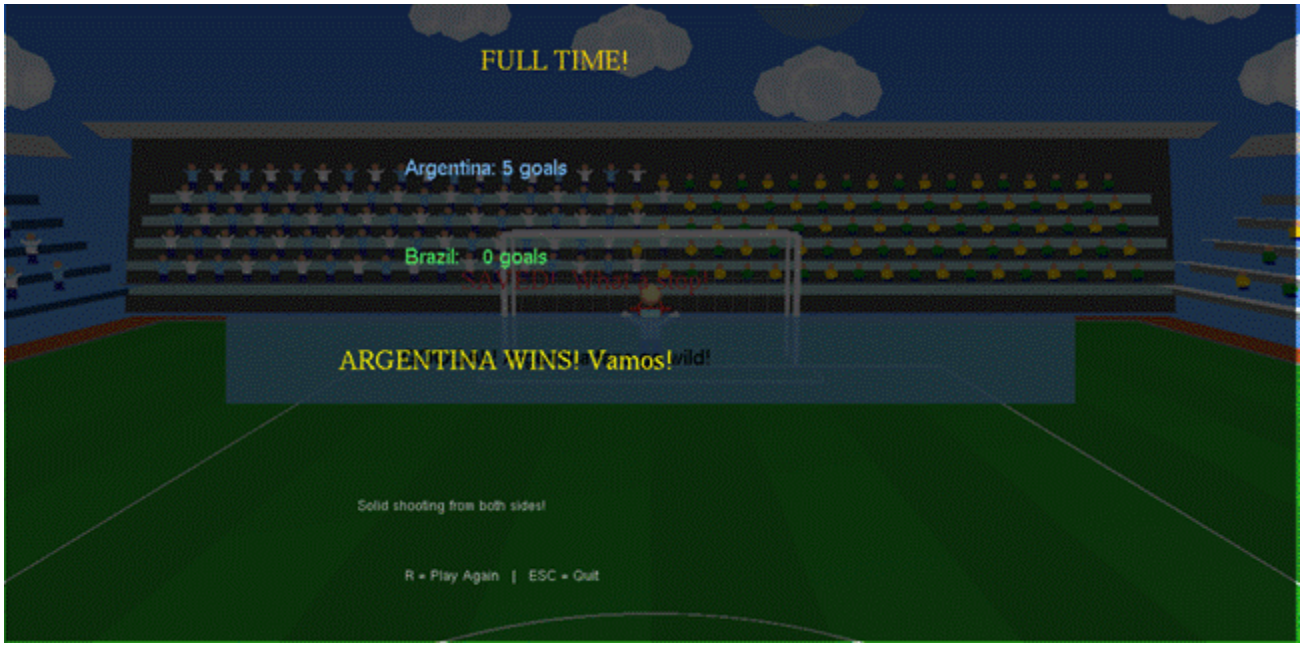
A / D	Playing (goalkeeper's turn)	Move keeper left/right	WASD convention; left hand stays on the keyboard naturally while right hand rests
LEFT / RIGHT	Playing (shooter's turn)	Aim crosshair left/right	Arrow keys provide fine-grained, satisfying directional control

8. Output.

8.1. MENU.




HALF TIME!
Argentina scored: 5 / 5
Now Brazil shoots! Argentina keeps!
P2 (Brazil) <- LEFT/RIGHT + SPACE
P1 (Argentina) <- A / D keeper
Press ENTER to start Round 2

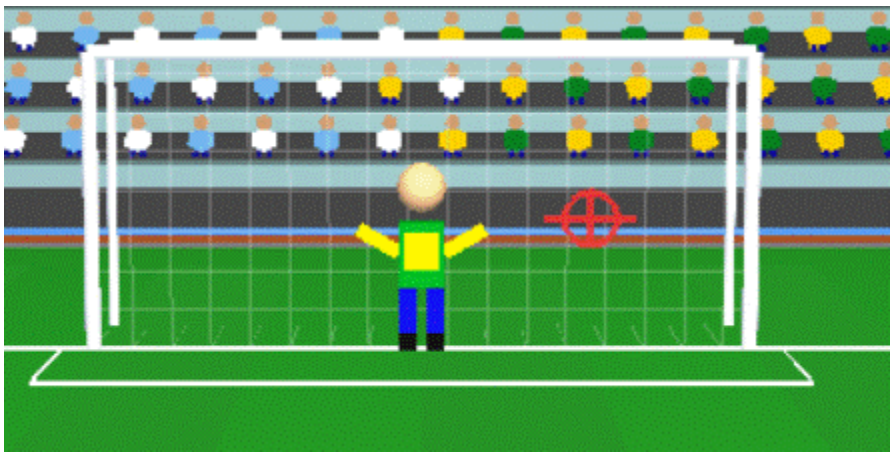

FULL TIME!
Argentina: 5 goals
Brazil: 0 goals
ARGENTINA WINS! Vamos!
Solid shooting from both sides!
R = Play Again | ESC = Quit

8.2. features.

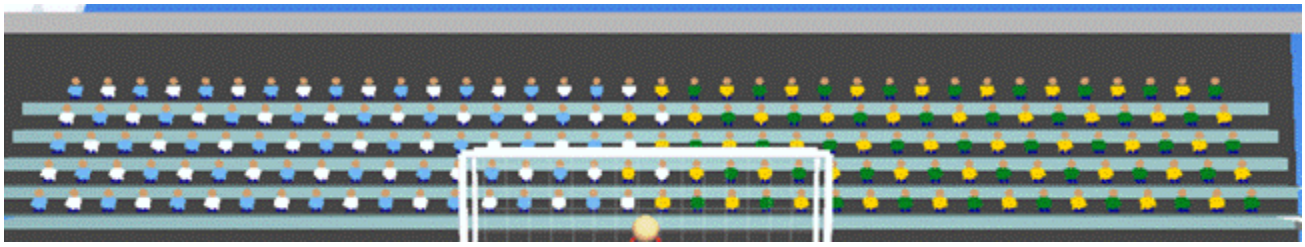
Cloud, bird, sun



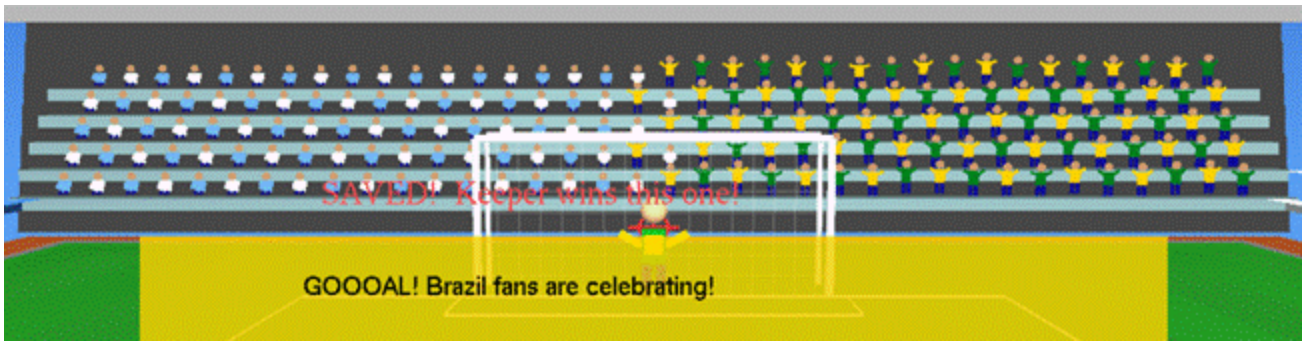
Goalpost and goalkeeper.

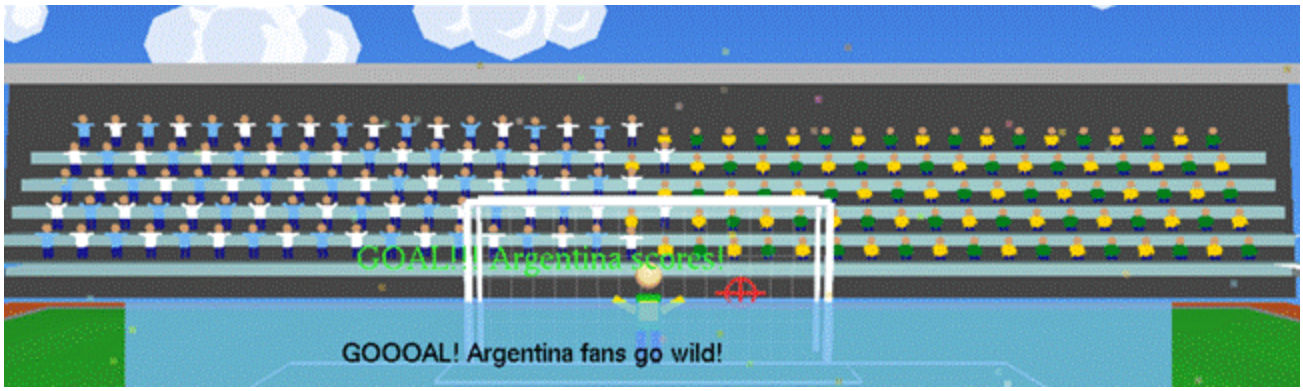


Crowd.



Crowed Celebrating of two team.



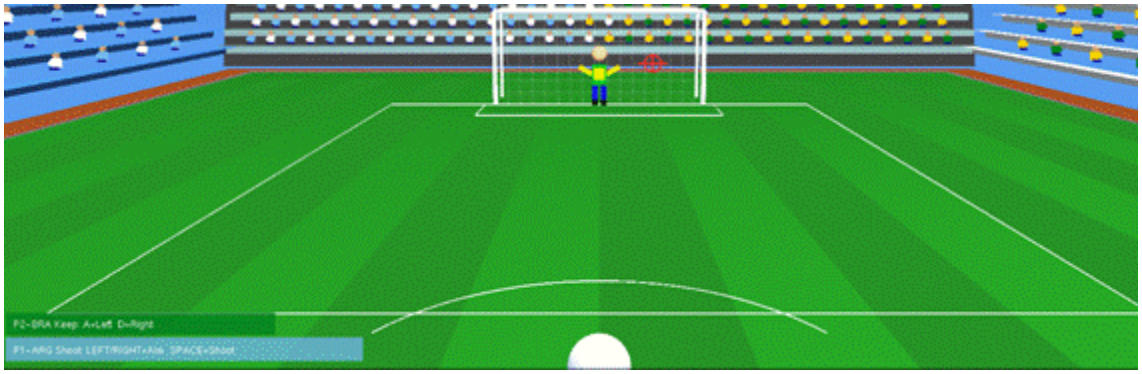


Live Statistics:

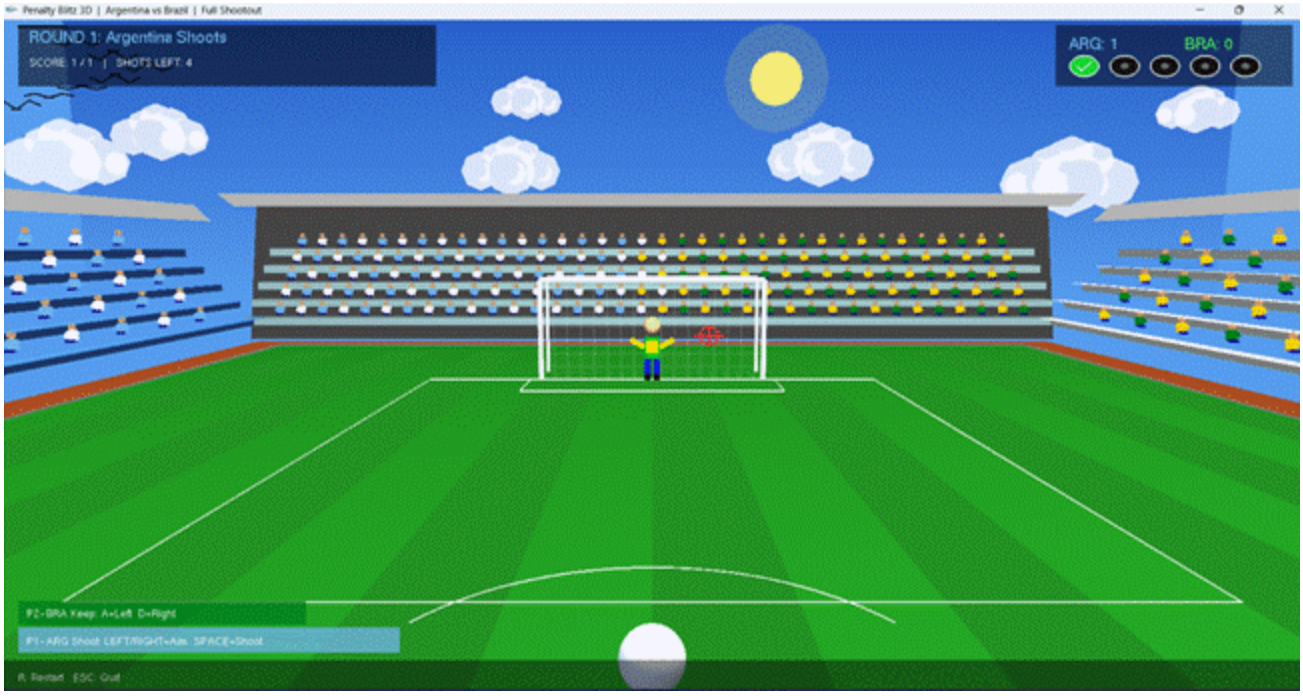




Penalty Shootout Area.



Full Stadium.



9. After Updating according to Dr.md.Tauhid Bin Iqbal (Assistant Professor) Sir Instruction ,

1.GoalKeeper Player Image

2. Player Shoot the ball

3.Goalpost Black Color

4.Pitch left right increase

Subject _____

Paper Sou

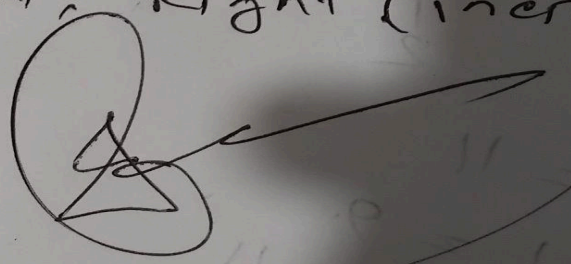
Date :____

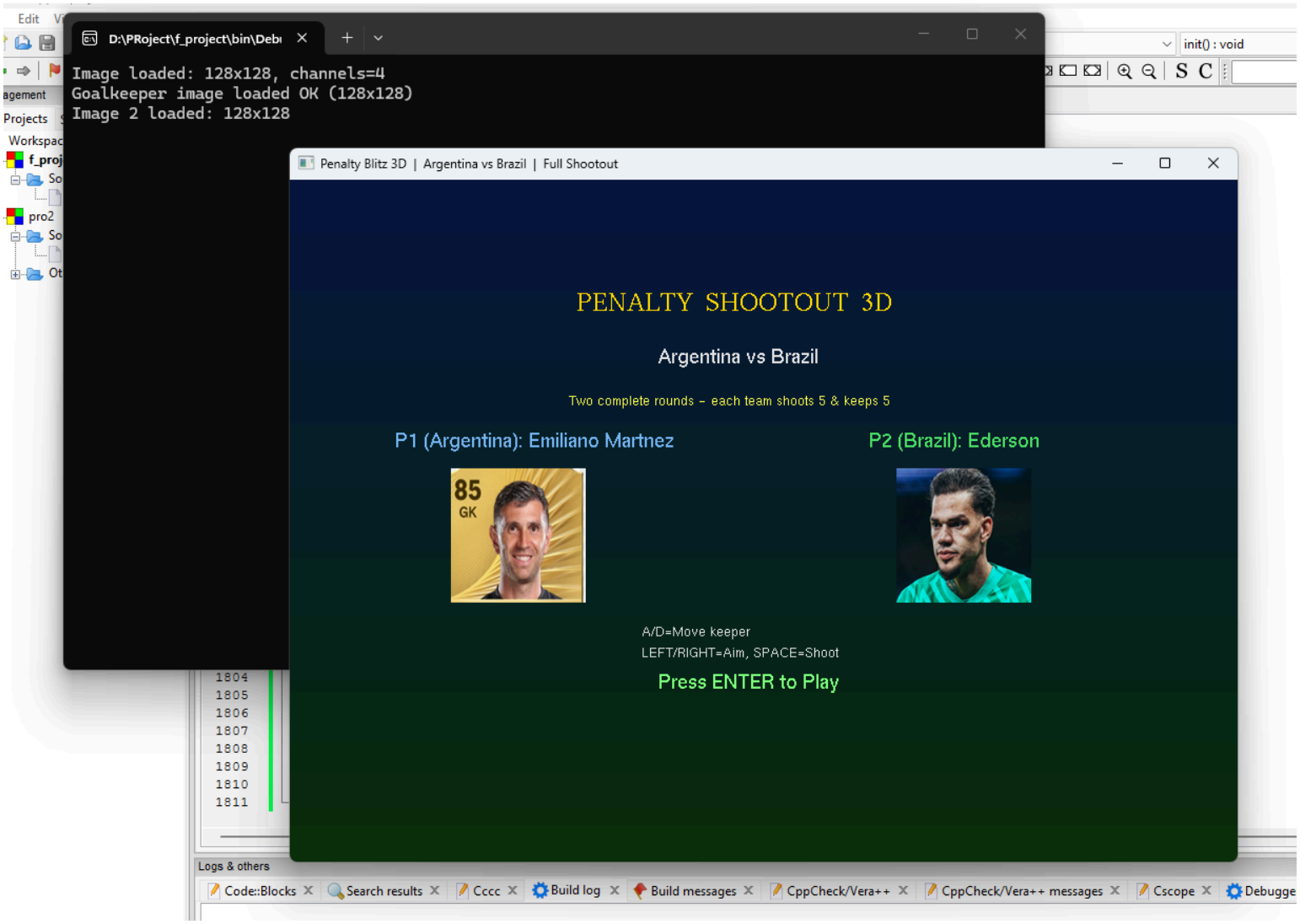
Goalkeeper — Player image.

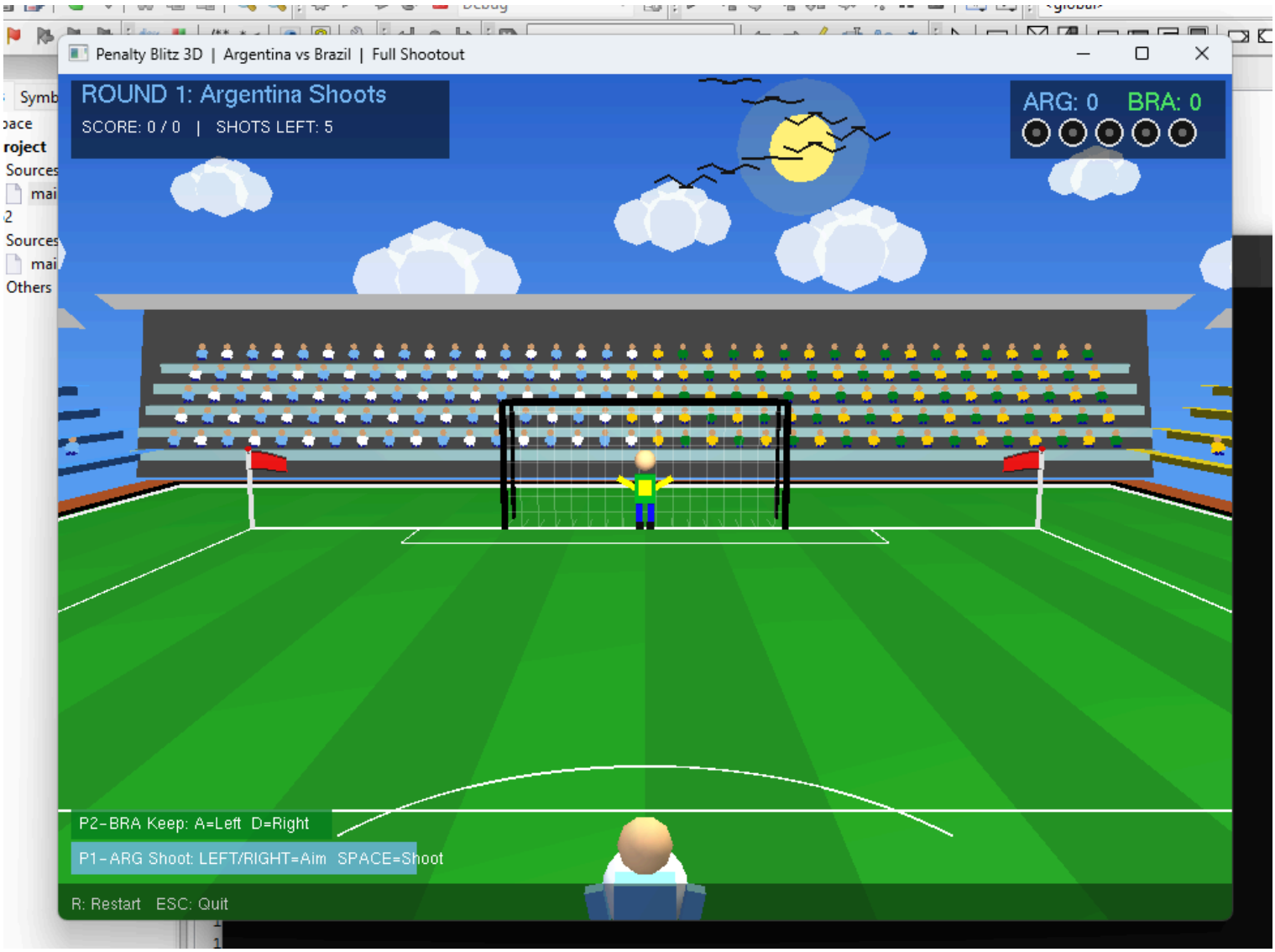
ball — Player shot.

Goalpost - Black color.

pitch — left - Right (increase).




D:\PRoject\f_project\bin\Deb
Image loaded: 128x128, channels=4
Goalkeeper image loaded OK (128x128)
Image 2 loaded: 128x128
Penalty Blitz 3D | Argentina vs Brazil | Full Shootout
PENALTY SHOOTOUT 3D
Argentina vs Brazil
Two complete rounds – each team shoots 5 & keeps 5
P1 (Argentina): Emiliano Martnez
P2 (Brazil): Ederson
85
GK
A/D=Move keeper
LEFT/RIGHT=Aim, SPACE=Shoot
Press ENTER to Play
Logs & others
Code::Blocks
Search results
Cccc
Build log
Build messages
CppCheck/Vera++
CppCheck/Vera++ messages
Cscope
Debugge


Penalty Blitz 3D | Argentina vs Brazil | Full Shootout
ROUND 1: Argentina Shoots
SCORE: 0 / 0 | SHOTS LEFT: 5
ARG: 0 BRA: 0
P2-BRA Keep: A=Left D=Right
P1-ARG Shoot: LEFT/RIGHT=Aim SPACE=Shoot
R: Restart ESC: Quit

10. Conclusion and Future Work

10.1 Conclusion

Penalty Blitz 3D successfully demonstrates the construction of a complete, interactive 3D sports simulation using only the C++ standard library, OpenGL's fixed-function pipeline, and FreeGLUT. The project achieves its core objectives: a visually rich stadium environment rendered entirely from geometric primitives, a physically motivated ball trajectory system, responsive two-player keyboard controls, a fully functional multi-round game loop, and a suite of atmospheric animations that elevate the scene beyond a static technical demonstration.

From a computer graphics standpoint, the project provides a clear, working example of perspective projection, Phong lighting with multiple sources, depth-tested transparency, immediate-mode geometry submission, and orthographic HUD overlays — all managed within a single, readable codebase. The modular function architecture makes the code highly accessible for study and extension.

From a game design perspective, the control scheme achieves genuine competitive tension: both players must commit to decisions under time pressure (the keeper must pre-position; the shooter must aim while the keeper is moving), making each of the ten shots in a full match a meaningful contest of prediction and reaction.

10.2 Limitations

- The absence of texture mapping limits visual realism; the pitch, crowd, and stadium rely entirely on flat colours and lighting for differentiation.
- The goalkeeper dive destination is implicitly the keeper's last position before the shot, rather than a computed reaction to the ball's trajectory, which reduces goalkeeper intelligence.
- No audio feedback (crowd roar, ball strike sound) is present, which significantly limits immersion.
- The game supports only local two-player mode on a shared keyboard, with no AI opponent option for single-player play.
- The fixed camera cannot be moved, preventing the player from appreciating the stadium from different angles.

10.3 Future Work

The following enhancements are proposed as directions for future development:

27. Texture Mapping: Apply grass, jersey pattern, and concrete textures using OpenGL's glTexImage2D to dramatically improve visual realism.

28. Modern Shader Pipeline: Port the rendering to OpenGL 3.3+ with GLSL vertex and fragment shaders, enabling per-pixel lighting, shadow mapping, and normal mapping.

29. AI Goalkeeper: Implement a machine learning or heuristic-based AI goalkeeper that learns from the human shooter's patterns over successive shots.

30. Audio Integration: Integrate OpenAL or SDL_mixer to add crowd ambient sound, kick impact audio, and celebration audio.

31. Vertical Aim Control: Extend the shooter's aim from one degree of freedom (X-axis only) to two (X and Y axes) for a more challenging and realistic shooting experience.

32. Network Multiplayer: Implement a client-server network layer to allow two players to compete over a local area network or the internet.

33. 3D Mesh Import: Replace the primitive-based character models with OBJ or FBX mesh imports for high-fidelity player and ball models.

34. Replay System: Record shot trajectories and goalkeeper positions during each round and play back a highlight reel on the result screen.

10.4 Final Remarks

Penalty Blitz 3D stands as a complete, self-contained demonstration of applied real-time 3D graphics programming. It bridges the gap between abstract computer graphics theory and a tangible, engaging interactive experience — making it a valuable project both as an educational artefact and as a foundation for more advanced future development. The careful attention to visual detail (animated fans, waving flags, atmospheric sky, particle effects) within the constraints of a legacy OpenGL API reflects the depth of creative and technical problem-solving that real-time graphics programming demands.